# LESSON 27

# 停下 跳过

break / continue

## break 和 continue：让循环听指挥

学完你能让循环在中途停下，或者跳过这一次，写出读到“再见”才结束的聊天程序。

# # 找到了，还在接着找?

第 25 课的鸡兔同笼

**找到了还试完**

第 22 课一直问到输对

**要记着 done**

循环原来只会从头转到尾。有时我们想在中间就停下：找到了就别再找，输对了就别再问。这节课学两个词：break 是“跳出”，整个循环马上结束；continue 是“继续”，跳过这一次，直接进下一圈。

# break：马上跳出循环

break.py

```python
for i in range(1, 10):
    if i * i > 30:
        break
    print(i)
print("停在", i)
```

```
1
2
3
4
5
停在 6
```

i 是 6 时，36 超过 30，运行 break：循环马上结束，7、8、9 都不再试。跳到循环后面那一行。

# 猜一猜：print 挪到 if 前面，会输出 6 吗？

break.py

```python
for i in range(1, 10):
    print(i)
    if i * i > 30:
        break
```

```
1 2 3
4 5 6
```

会输出 6。i 是 6 时，先运行 print(6)，再检查 36 > 30，然后 break。
break 只管它后面的事：它前面已经运行过的，不会收回。

# 小测：这段程序输出几？

jump.py

```python
n = 0
while True:
    n = n + 3
    if n > 10:
        break
print(n)
```

1 9

2 10

3 12

4 一直转下去

答对了！加到 12 才超过 10，break 跳出，输出 12。

# 例题：说“再见”才停

chat.py

```python
while True:
    word = input()
    if word == "再见":
        break
    print("你说", word)
print("拜拜")
```

输入“再见”，break 跳出循环，最后说“拜拜”。

```
你说 你好
你说 吃了吗
拜拜
```

# 第 22 课的例题，换个写法

retry.py

```python
while True:
    try:
        age = int(input())
        break
    except ValueError:
        print("不是整数，再输一次")
print("明年你", age + 1, "岁")
```

转换成功，才走到 break；出错就跳到 except，break 被跳过，接着再问。

不用 done 记着，读起来就像在说：一直问，输对了就停。

依次输入 abc、3.5、12：

```
不是整数，再输一次
不是整数，再输一次
明年你 13 岁
```

# continue：跳过这一次

skip.py

```python
for i in range(1, 11):
    if i % 3 == 0:
        continue
    print(i)
```

```
1 2
4 5
7 8
10
```

i 是 3 时运行 continue：剩下的 print 不做了，直接去下一圈。6 和 9 也被跳过。

break 是整个循环结束，continue 只跳过这一次。

# 小测：continue 换成 break，输出什么？

1 1 1 2

2 1 2 4 5 7 8 10

3 3 6 9

4 什么也不输出

答对了！i 是 3 时 break，整个循环结束，只输出了 1 和 2。

# 动手改一改

把聊天程序改成记账。

## 读到 0 就停，算总和

在例题上改：每行读入一个整数，读到 0 就停。停下以后输出前面所有数的和，0 不算。不知道一共几个数，所以用 while True。

依次输入 5、8、-3、0

```
10
```

第一个就是 0

```
0
```

输入要先变成整数，总和从 0 开始。教室电脑：打开 starter/05_sum_until_zero.py

# 这节课常见的三种错误

划线的写法只阅读，不要照抄

| | | |
|---|---|---|
| break 写在循环外 | SyntaxError | break 只能用在循环里 |
| while True 没写 break | 停不下来 | 要有一个能走到的出口 |
| 该停处写成 continue | 停不下来 | continue 只跳过这一次 |

写 while True 时先问自己：哪一句 break 能让它停下？那句真的走得到吗？

# 找错：说了“再见”，程序还在等着输入

bug.py（只阅读）

```
1 count = 0
2 while True:
3     word = input()
4     if word == "再见":
5         continue
6     count += 1
7 print("说了", count, "句")
```

想数一数说了几句，
可它一直停不下来。
点出写错的那一行。

点代码里的一行

# 自己挑战

# 整理成绩单

不停读入分数，每行一个整数，读到 -1 就结束。不在 0 到 100 之间的是写错了，跳过不算。
输出两行：“有效 几 个”和“平均 多少”，平均分用 round 保留 1 位小数。一个有效分数也没有，就输出“没有有效分数”。

依次输入

```
90
80
120
85
-1
```

输出

```
有效 3 个
平均 85.0
```

先自己写，卡住了再翻下一页。

# 卡住了？一次看一条提示

**提示 1**　用 while True 一直读。读到 -1 用 break 停下，和动手题读到 0 一样。

**提示 2**　小于 0 或者大于 100 的，用 continue 跳过：不加进总和，也不数。

**提示 3**　循环结束后再算平均。个数是 0 时不能除，先判断，说“没有有效分数”。

# 造一个“知道什么时候停”的小程序

用 break 让它在该停的时候停，用 continue 跳过不要的。

## 先做到

改一改聊天程序：换一句结束的话，再换一种回答。

## 选你喜欢的

密码锁：最多试 3 次；点菜机：说“结账”才算总价；跳过不爱吃的菜。

## 还想更厉害

把第 21 课的猜数字改成 while True，再数一数猜了几次。

没有标准答案。先说清它要做什么，再试两组输入，一组试边界。

# # 和同桌换着玩

## 1

### 换着玩

玩同桌的程序，试试它能不能按说好的那样停下

## 2

### 说一个喜欢的

告诉同桌，哪里设计得最好玩

## 3

### 提一个想法

有没有一种输入，会让它停不下来？

听完再改一版，让程序更稳。

# 说得清吗?

| | | |
|---|---|---|
| Q1 | **break 做什么?** | 马上跳出整个循环，去运行循环后面的。 |
| Q2 | **continue 做什么?** | 跳过这一圈剩下的，直接进下一圈。 |
| Q3 | **while True 什么时候停?** | 这节课靠里面的 break 停，所以要保证 break 一定走得到。 |

# 速查卡

| | |
|---|---|
| `break` | 整个循环马上结束 |
| `continue` | 跳过这一次，进下一圈 |
| `while True:` | 一直转，靠里面的 break 停 |
| `if word == "再见":` | 先判断，再 break |
| break 前面的已经做了 | 不会收回 |

# # 这些后面再学

### 两层循环里的 break

只跳出它所在的那一层

### for ... else

循环没被 break 打断时，才运行 else

### pass

什么也不做，先占个位置

今天学会了让循环听指挥：该停就停，该跳就跳。

# 谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

 python-for-youth

下一课：f-string 格式化输出